

# Theme: AI for Smart Mobility

## Scenario-Based Challenge

Transportation and logistics systems are becoming increasingly complex as cities grow, delivery networks expand, and commuters expect faster, safer, and more reliable mobility services. However, organizations and individuals often rely on disconnected tools, static planning methods, and manual decision-making, resulting in inefficient routes, increased operational costs, traffic congestion, delivery delays, poor fleet utilization, and suboptimal commuter experiences.

Design an **AI-powered Smart Mobility solution** that addresses a real-world transportation or logistics challenge by intelligently optimizing the movement of people, vehicles, goods, or services. Solutions should leverage Artificial Intelligence to analyze real-time and historical data, provide personalized recommendations, predict disruptions, optimize routes and schedules, improve fleet efficiency, enhance commuter experiences, and support smarter mobility decisions.

Participants are encouraged to identify a meaningful mobility problem and develop an innovative AI-driven solution rather than implementing a predefined use case.

---

## REQUIREMENTS

### Submission Requirements

Participants must submit the following:

- **Problem Statement** – A brief description of the problem being addressed.
- **Solution Description** – An overview of the proposed solution, including how AI is integrated and the key features of the application.
- **GitHub Repository** – A public GitHub repository containing the complete frontend and backend source code, along with a README explaining the project setup and execution steps.
- **Deployed Application Link** – A publicly accessible deployment of the application.
- **Demo Video** – A short video (3–5 minutes) demonstrating the application's features, AI capabilities, and overall workflow.

---

# ASSIGNMENT TECH STACK

## Frontend

- React.js
- Vite
- React Router
- Tailwind CSS, Material UI, Bootstrap, or another React UI framework
- Axios or Fetch API
- Maps integration (Google Maps, Mapbox, Leaflet, or similar) (*optional*)

## Backend

- Node.js
- Express.js
- JSON Web Token (JWT) authentication
- bcrypt for password hashing
- Zod, Joi, or Express Validator

## Database (Choose Any One)

- MongoDB
- PostgreSQL
- MySQL
- SQLite
- Supabase PostgreSQL

## Artificial Intelligence

- Google Gemini API
- OpenAI API
- Anthropic Claude API

The AI API key must be stored only in backend environment variables.

---

# TOOLS TO USE

## IDE'S

- Visual Studio Code IDE
- Cursor
- Antigravity

## Version Control

- Git
- GitHub

## Recommended

- Postman or Bruno for API testing
- 

# DEPLOYMENT

## Frontend

- Vercel or Netlify

## Backend

- Render, Railway, or Fly.io

## Database

- MongoDB Atlas
- Supabase
- Neon
- Railway PostgreSQL

## Deployment Guide:

[https://github.com/kellatirupathi/Careerpilot-ai/blob/main/DEPLOYMENT\\_GUIDE.md](https://github.com/kellatirupathi/Careerpilot-ai/blob/main/DEPLOYMENT_GUIDE.md)

